

NODE_PROVISIONING_BOUNDARY

Helix Cluster OS — Node-Provisioning Boundary

Field	Value
Revision	1
Created	2026-06-03
Status	active

HXC-1146. This document is the canonical, normative statement of the node-provisioning **boundary** for Helix Cluster OS. It defines, without ambiguity, what the project does and does **not** do when a node is brought into the cluster. It is binding on every component that provisions, flashes, onboards, enrolls, or attests a node.

1. Normative Statement

Helix Cluster OS provisions and onboards nodes that the operator already controls and has lawful administrative access to. It does NOT defeat, bypass, or weaken any device's security model.

The project's provisioning surface — building agent images, flashing operator-owned boards, side-loading the agent onto operator-controlled devices, enrolling owned servers, mesh enrollment, attestation, and trust admission — begins **after** the operator has lawful root/admin control of the device. The act of *obtaining* that control is **out of scope** and is the operator's own, independently-acquired responsibility.

The verbs **MUST**, **MUST NOT**, **SHALL**, and **SHALL NOT** in this document are normative (RFC 2119 sense).

2. OUT OF SCOPE (the project **MUST NOT** implement)

Helix Cluster OS **MUST NOT** implement, ship, bundle, document-as-a-feature, or automate any of the following, because each requires defeating a device's security model:

- **Jailbreaking** — escaping a vendor-locked OS sandbox (e.g. consoles, locked mobile OSes).
- **Rooting** — privilege-escalation exploits to obtain administrative access the device's owner has not been granted by lawful means.
- **Bootloader unlocking via exploit** — defeating a locked or signed bootloader / verified-boot / secure-boot chain.
- **Carrier / SIM / network unlock** — removing carrier or operator locks.
- **DRM / signature / fuse / TPM bypass** — circumventing code-signing, e-fuse, secure-enclave, or attestation enforcement that the device vendor put in place.
- **Any OS- or security-bypass** — any technique whose purpose is to obtain access that a device's security model is designed to deny.

A pull request, package, script, or document that adds any of the above is a boundary violation and a release blocker. There is **no** flag, build tag, or “research mode” that re-enables it.

3. IN SCOPE (the project **DOES** implement)

Helix Cluster OS provisions nodes the operator **already controls**. In scope:

- **Flashing operator-owned SBCs.** Building and writing Helix node images to single-board computers the operator physically owns and is free to reflash (e.g. ARM64 SBCs), using the board's normal, vendor-sanctioned flashing path.
- **Side-loading the agent on operator-controlled devices.** Installing the Helix node agent onto devices where the operator already has lawful admin rights and the platform permits app/binary installation (e.g. side-loading the agent build on an Android device the operator administers, installing on a developer-unlocked handheld the operator owns).
- **Enrolling owned servers and workstations.** Onboarding datacenter GPUs, servers, and workstations the operator owns or is lawfully authorized to administer.
- **Cluster onboarding for already-controlled nodes** — mesh enrollment (WireGuard), membership (SWIM gossip), service discovery, GPU/resource attestation, and trust admission. These all operate on a node to which the operator already holds admin access.

Related roadmap work items: ARM64 cross-compile / toolchain and SBC node agent provisioning, and the Android / edge-mobile agent track (see *Phase 3 — Edge/mobile agents (ARM64, Android, iOS)* in [docs/research/PHASE_2_ROADMAP.md](#)). Each builds and installs an agent for a node the operator already controls; none unlocks a device.

4. Lawful-Control Precondition

Every in-scope provisioning path assumes — and the operator warrants — that the operator has **lawful administrative control** of the target node:

1. The operator **owns** the device, or is **explicitly authorized** by the owner to administer it; and
2. Admin/root access was obtained by **lawful, vendor-sanctioned means** (the operator flashed it, the device shipped operator-administered, the operator was granted credentials, the device is developer-unlocked by its own owner), **not** by any technique in §2.

Helix Cluster OS treats this precondition as already satisfied at the point of provisioning. It neither provides nor automates the means to satisfy it by bypassing a device's protections.

5. Rationale — Security & Trust Model

Helix's security model is **trust-by-attestation among operator-controlled nodes**, not trust-by-conquest of locked devices:

- **Admission requires attestation, not exploitation.** A node joins via cryptographic identity (SPIFFE federation), mesh keys (WireGuard), and hardware/GPU attestation challenge/response. These prove *what* a node is to the cluster; they presuppose the operator already administers it. They are not, and must never become, a means of seizing a device.
- **The bypass surface is a non-goal and a liability.** Shipping unlock/exploit tooling would expand the attack surface, undermine the attestation chain, and create legal and supply-chain exposure entirely orthogonal to the project's purpose (orchestrating compute across nodes operators already run).
- **Clarity for contributors and reviewers.** Code review (CLAUDE-1) and the governance rules treat any device-security-bypass contribution as a first-class defect. This document is the authoritative reference reviewers cite when rejecting such work.

In short: **we provision the nodes you control; we do not unlock the ones you don't.**

6. Cross-References

- [README.md](#) — project overview and Documentation index.
- [CLAUDE.md](#) — AI-agent engineering & governance rules (CLAUDE-1 usability, CLAUDE-2 cross-platform parity, CLAUDE-3 docs-sync).
- [docs/ARCHITECTURE.md](#) — L0–L7 architecture; mesh, membership, discovery, and attestation components referenced above.
- [docs/research/PHASE_2_ROADMAP.md](#) — console / edge node integration context and the edge/mobile agent track.